

✓ AI534 Warm up exercises 0

This is a warm-up assignment (individual) for you to get familiar with some basics:

1. Using google colab and python notebook to complete implementation assignments
2. Basic packages and functions for working with data, performing simple analysis and plotting.
3. Walk you through some such basic steps for getting an intuitive understanding of what your data looks like, which is the first step to tackling any machine learning problem.

We will use a data set that contains historic data on houses sold between May 2014 to May 2015. Each house in the data set is described by a set of 20 descriptors of the house (referred to as features or attributes, denoted by x mathematically) and tagged with the selling price of the house (referred to as the target variable or label, denoted as y).

Let's get started by importing the necessary packages.

```
!pip install nbconvert > /dev/null 2>&1
!pip install pdfrkit > /dev/null 2>&1
!apt-get install -y wkhtmltopdf > /dev/null 2>&1
import os
import pdfrkit
import contextlib
import sys
from google.colab import files
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

✓ Follow along step 1: accessing and loading the data

First, you need to download the file ia0_train.csv (provided on canvas) to your google drive. To allow the colab to access your google drive, you need to mount Google Drive from your notebook:

```
from google.colab import drive
drive.mount('/content/gdrive')
```



Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

Set the path to the data file:

```
file_path='/content/gdrive/MyDrive/ia0_train.csv' #please use the same path to store your data file to avoid needing modification to run your code
```

Now load the csv data into a DataFrame, and take a look to see what it looks like.

```
raw_data = pd.read_csv(file_path)
raw_data
```

```
#you can see the data type for each column
raw_data.dtypes
```

> Follow along step 2: Understanding and preprocessing the data

As you can see from the output of the previous cell, there are 10k examples, each with 21 columns in this csv file. The column 'price' stores the price of the house, which we hope our model can learn to predict. The other columns are considered the input features (or attributes). Before feeding this data to a machine learning algorithm to learn a model, it is always a good idea to examine the features, as **features are not always useful** and also they might be in a format that is not well suited for our learning algorithm to consume. Here are two immediate issues in this regard:

1. The ID feature is a unique identifier assigned to each example, hence it carries no useful information for generalization and should not be included as a feature for machine learning. You should drop this column from the data before feeding to the learning algorithm.
2. The date feature is currently in the object format, which means it is string. Most of ML algorithms assume numerical inputs, hence we want to change it into a numerical feature. Here please break the date string into three separate numerical values representing the Month, day and year of sale respectively.

Follow along step 3: check out some specific features

The first thing coming to mind when buying a house is the number of rooms, bedrooms, bathrooms, these are going to be among the most important factors deciding the price of a house. So let's check these features out. Specifically, let's take a look at the statistics of these features.

```
# Group the data by the 'bedrooms' column and calculate statistics for 'price'
bedroom_stats = data_without_id.groupby('bedrooms')['price'].agg(['mean', 'median', 'min', 'max', 'count'])
bedroom_stats

# Group the data by the 'bathrooms' column and calculate statistics for 'price'
bathroom_stats = data_without_id.groupby('bathrooms')['price'].agg(['mean', 'median', 'min', 'max', 'count'])
bathroom_stats
```

You can see there are a lot more unique values than one might expect (what is .75 bathroom? I wonder about that too). Now to verify our intuition that more bedroom and bath room leads to higher pricing, we can further visualize the price distribution for each bedroom and bathroom number. This can be achieved by grouping price data by the different values of bedrooms, and bathrooms, then use box plots to visualize how prices are distributed, given specific values for the numbers of bedrooms / bathrooms:

```
# find the unique number of bedrooms in the data
unique_bedrooms = sorted(data_without_id['bedrooms'].unique())

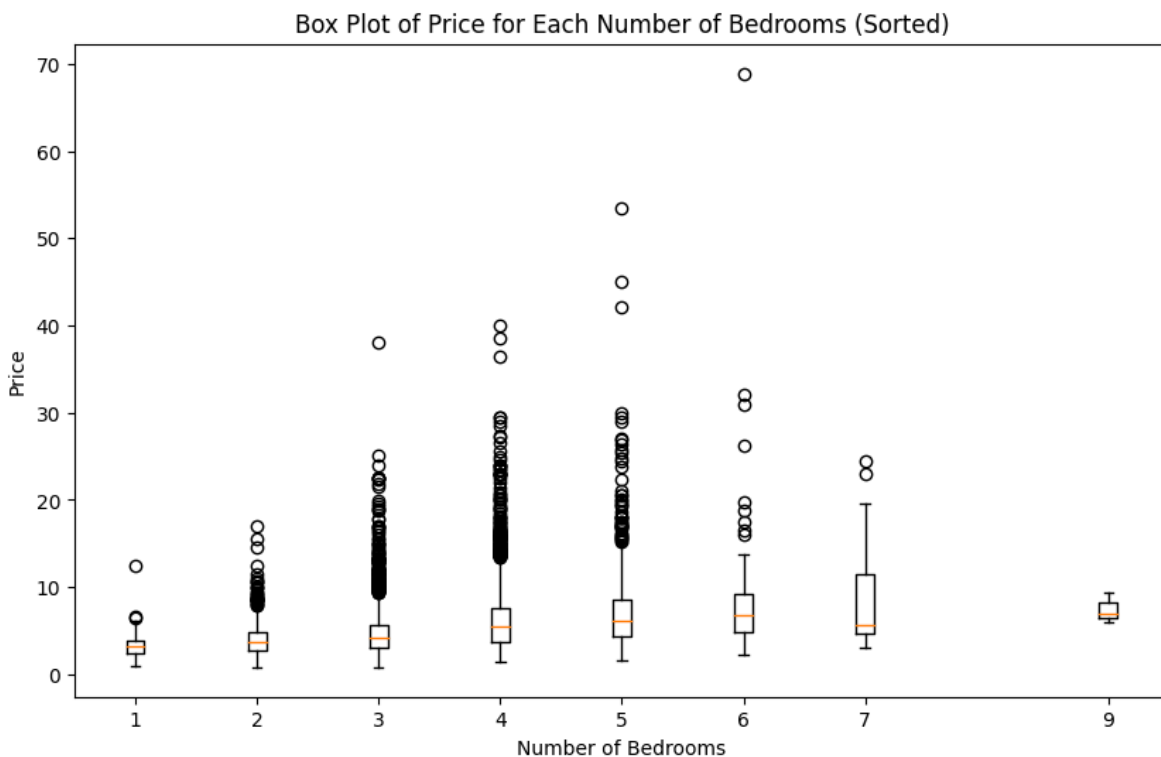
# Create a box plot of 'price' for each unique number of bedrooms with at least 3 examples
plt.figure(figsize=(10, 6)) # Adjust the figure size if needed

for num in unique_bedrooms:
    bedroom_data = data_without_id[data_without_id['bedrooms'] == num]['price']

    # Skip plotting if there are less than 3 examples with this number of bedrooms. you can remove the skipping and see the effect.
    if len(bedroom_data) >= 3:
        plt.boxplot(bedroom_data, positions=[num], labels=[num], showfliers=True)

# Add labels and a title to the plot
plt.xlabel('Number of Bedrooms')
plt.ylabel('Price')
plt.title('Box Plot of Price for Each Number of Bedrooms (Sorted)')

# Show the plot
plt.show()
```



```
# find the unique number of bathrooms in the data
unique_bathrooms = sorted(data_without_id['bathrooms'].unique())

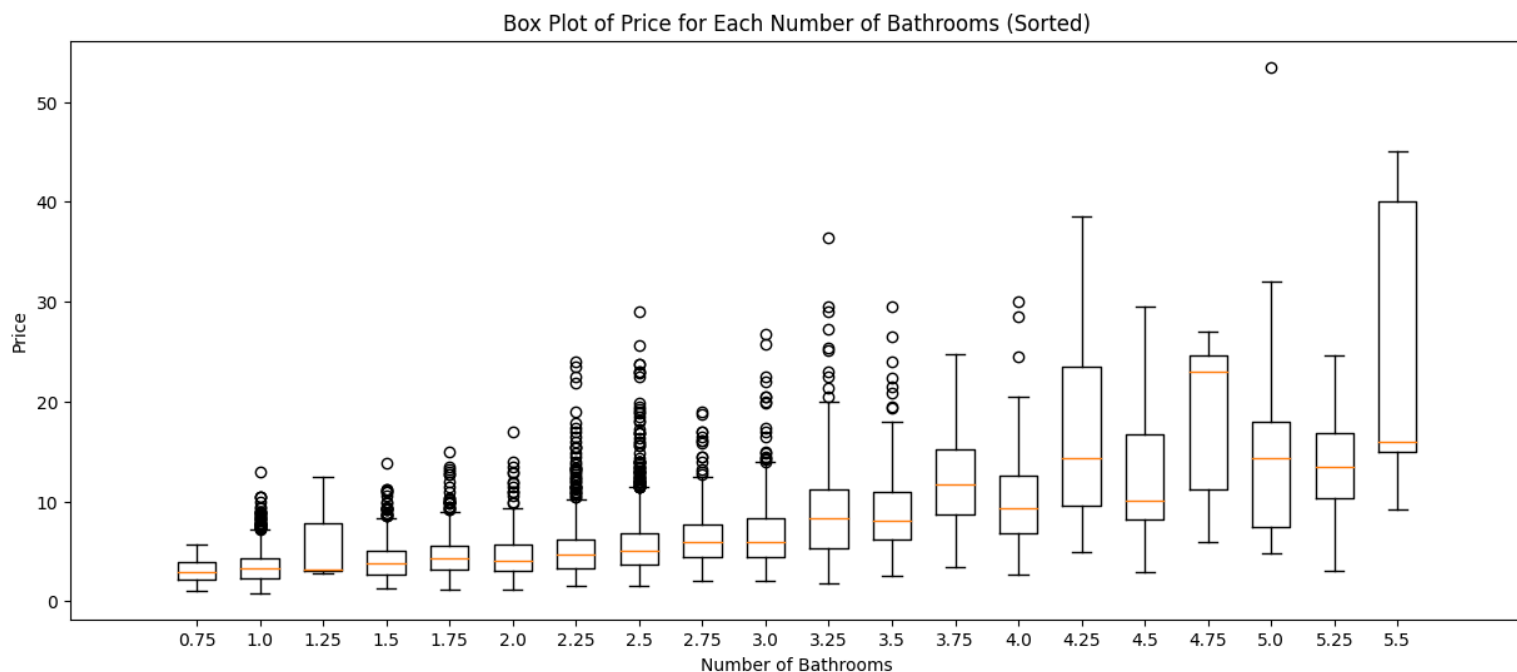
# Create a box plot of 'price' for each unique number of bedrooms with at least 3 examples
plt.figure(figsize=(15, 6)) # Adjust the figure size if needed

for num in unique_bathrooms:
    bathroom_data = data_without_id[data_without_id['bathrooms'] == num]['price']

    # Skip plotting if there are less than 3 examples with this number of bedrooms
    if len(bathroom_data) >= 3:
        plt.boxplot(bathroom_data, positions=[num], labels=[num])

# Add labels and a title to the plot
plt.xlabel('Number of Bathrooms')
plt.ylabel('Price')
plt.title('Box Plot of Price for Each Number of Bathrooms (Sorted)')

# Show the plot
plt.show()
```



As can be seen from the results above, the price does appear to adhere to the "more rooms -> more expensive" trend. We can also create a heatmap to show the price of the house as a function of the # of bathroom and # of bedrooms using the seaborn package.

```
import seaborn as sns

# Create a pivot table to prepare data for the heatmap
pivot_table = data_without_id.pivot_table(index='bedrooms', columns='bathrooms', values='price', aggfunc='mean')

# Create a heatmap using seaborn
plt.figure(figsize=(14, 10)) # Adjust the figure size if needed
heatmap = sns.heatmap(pivot_table, cmap='YlGnBu', annot=True, fmt='.2f', cbar=True)

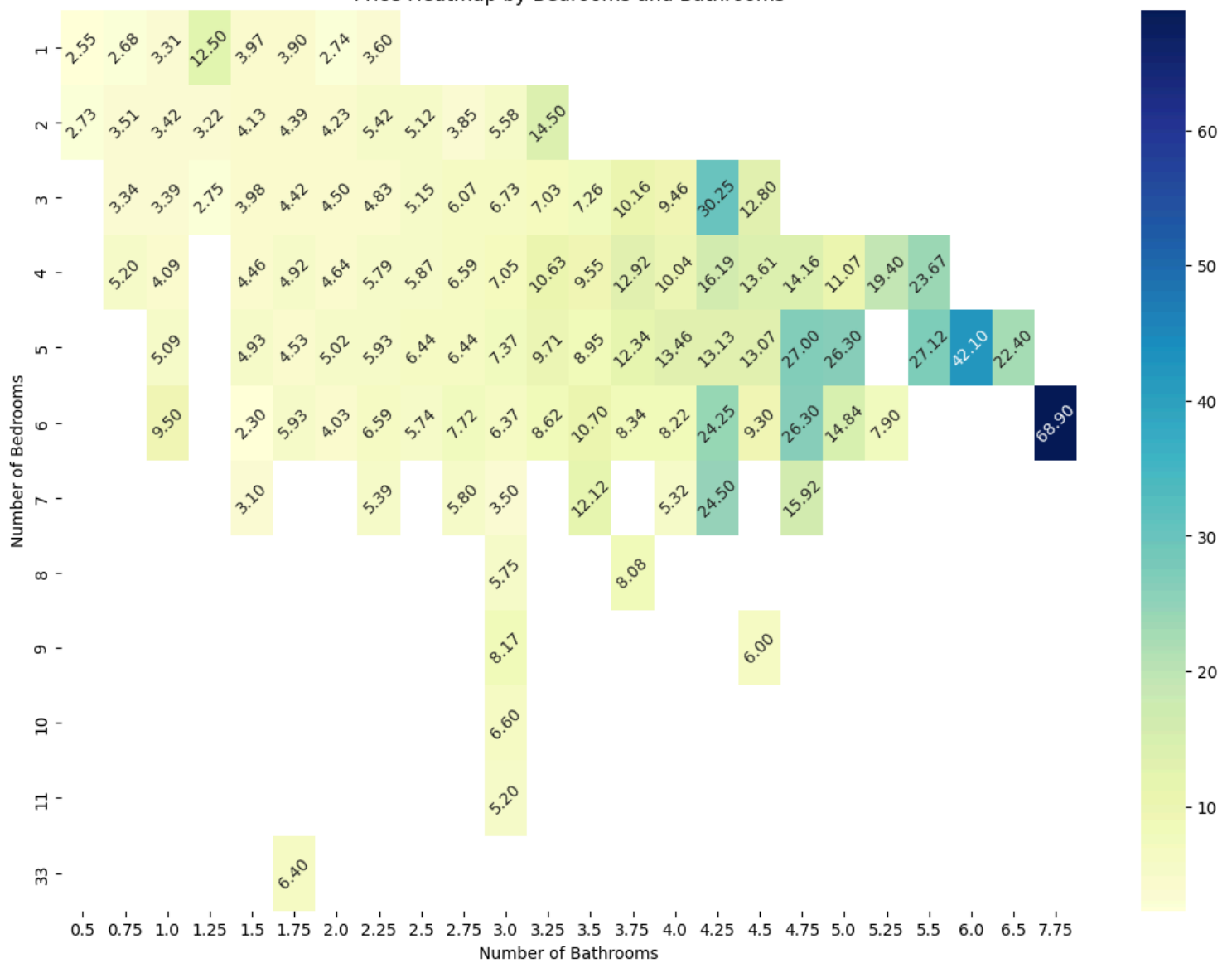
for text in heatmap.texts:
    text.set(rotation=45)

# Add labels and a title to the plot
plt.xlabel('Number of Bathrooms')
plt.ylabel('Number of Bedrooms')
plt.title('Price Heatmap by Bedrooms and Bathrooms')

# Show the plot
plt.show()
```

[4]

Price Heatmap by Bedrooms and Bathrooms



Does the trend follow your expectation? Any outliers?

The trend does not seem to follow my expectation that for a given number of bedrooms, the increase in the number of bathrooms would increase the house price. It does not follow a trend consistently. It is also observed that the price of houses for a given number of bedrooms is significantly high if the number of bathrooms just crosses the bedroom count. Outliers: For 33 number of bedrooms having 1.75 bathrooms is a significant outlier. Cases like the house having 6 bedrooms but 1 bathroom, having greater price than the most of its bedroom category.

Another intuitively important feature for a house is the square footage of the house. We can plot the price of the house against the square footage of the house and see if there is a clear trend as expected.

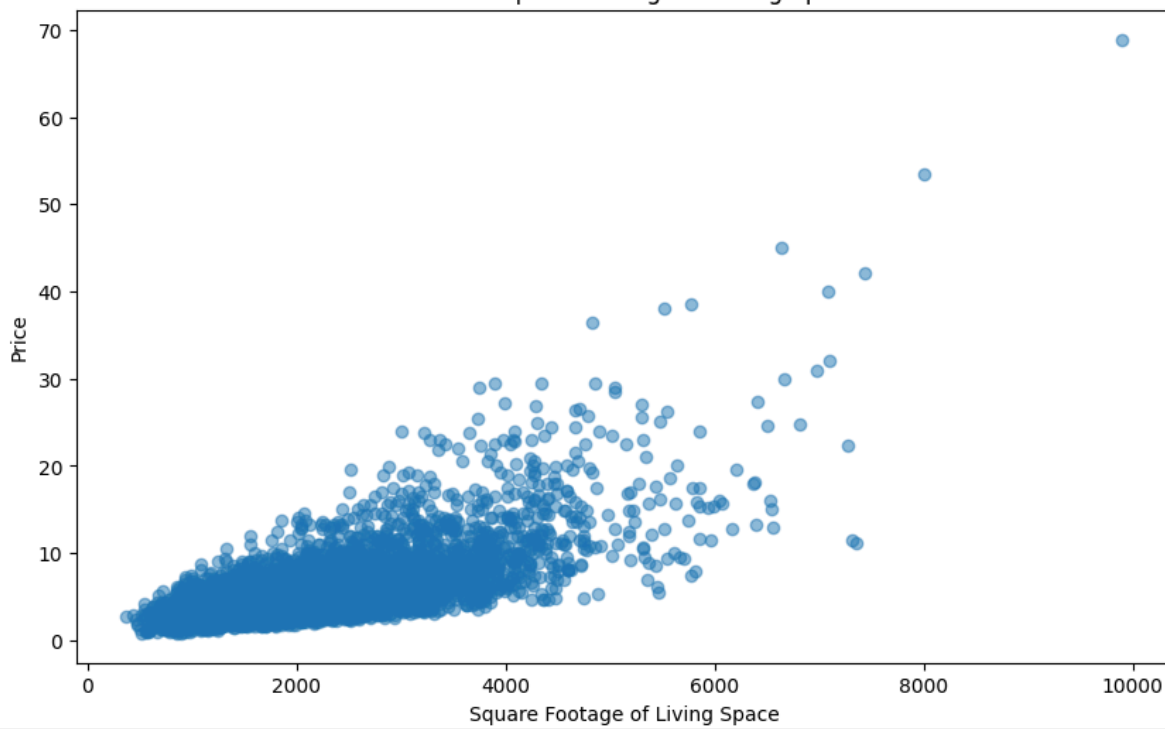
```
plt.figure(figsize=(10, 6)) # Adjust the figure size if needed
plt.scatter(data_without_id['sqft_living'], data_without_id['price'], alpha=0.5)

# Add labels and a title to the plot
plt.xlabel('Square Footage of Living Space')
plt.ylabel('Price')
plt.title('Price vs. Square Footage of Living Space')

# Show the plot
plt.show()
```



Price vs. Square Footage of Living Space



Closer inspection reveals that there are several features associated with square footage. Let's see how strongly correlated they are with one another.

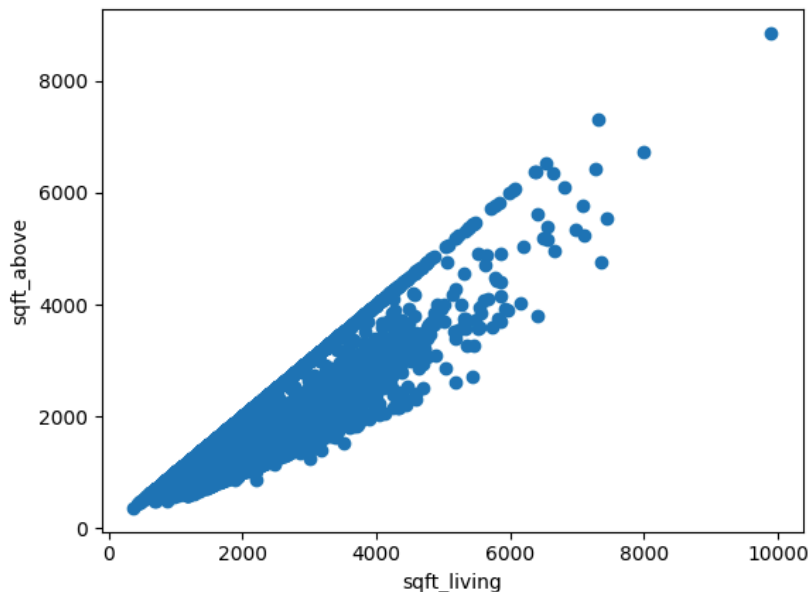
`data_without_id[["sqft_living", "sqft_lot", "sqft_living15", "sqft_lot15", "sqft_above", "sqft_basement", "yr_built", "yr_renovated", "zipcode", "grade", "flooded", "view"]]`



	sqft_living	sqft_lot	sqft_living15	sqft_lot15	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	grade	flooded
sqft_living	1.000000	0.164651	0.762189	0.178888	0.878699	0.416699	0.315164	0.040015	-0.193850	0.757528	0.354314
sqft_lot	0.164651	1.000000	0.139211	0.774133	0.176956	0.007146	0.047705	0.007068	-0.124270	0.101992	-0.013066
sqft_living15	0.762189	0.139211	1.000000	0.171446	0.737738	0.188109	0.327346	-0.012562	-0.277757	0.712580	0.279685
sqft_lot15	0.178888	0.774133	0.171446	1.000000	0.190612	0.010897	0.068022	0.001545	-0.140117	0.114872	-0.019085
sqft_above	0.878699	0.176956	0.737738	0.190612	1.000000	-0.067804	0.421716	0.012680	-0.259914	0.754622	0.521707
sqft_basement	0.416699	0.007146	0.188109	0.010897	-0.067804	1.000000	-0.144371	0.059483	0.089802	0.146215	-0.252958
yr_built	0.315164	0.047705	0.327346	0.068022	0.421716	-0.144371	1.000000	-0.223462	-0.346055	0.448443	0.488509
yr_renovated	0.040015	0.007068	-0.012562	0.001545	0.012680	0.059483	-0.223462	1.000000	0.058484	0.006410	0.004294
zipcode	-0.193850	-0.124270	-0.277757	-0.140117	-0.259914	0.089802	-0.346055	0.058484	1.000000	-0.189857	-0.059987
grade	0.757528	0.101992	0.712580	0.114872	0.754622	0.146215	0.448443	0.006410	-0.189857	1.000000	0.465624
flooded	0.354314	-0.013066	0.279685	-0.019085	0.521707	-0.252958	0.488509	0.004294	-0.059987	0.465624	1.000000
view	0.281220	0.078483	0.276776	0.064136	0.172350	0.259540	-0.052253	0.091621	0.072525	0.247440	0.035503

Sqft_living and sqft_above are the two most correlated feautres. We can visualize their relationship by using a scatter plot:

```
plt.scatter(data_without_id['sqft_living'].values, data_without_id['sqft_above'].values)
plt.xlabel("sqft_living")
plt.ylabel("sqft_above")
plt.show()
```



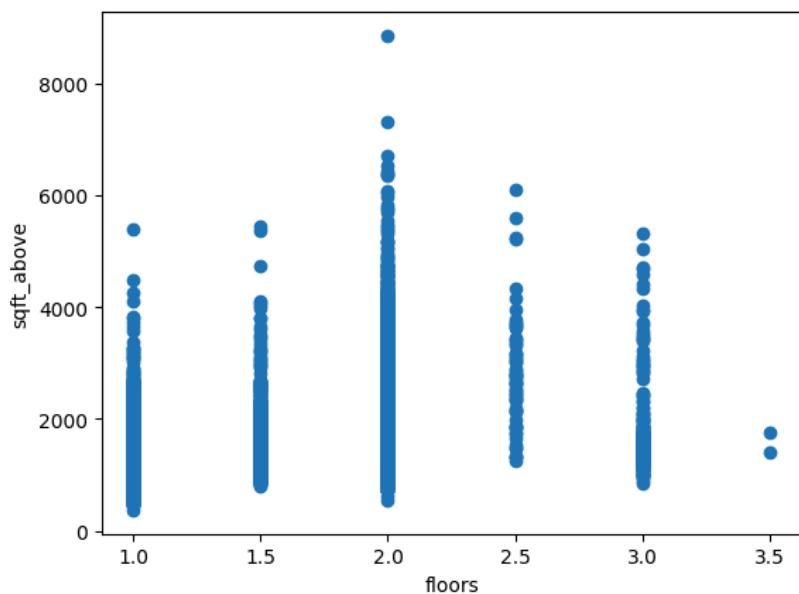
When we have features that are highly redundant, it is important to understand the impact of such redundant features to the learning algorithm. We will explore more on this in later assignments.

✓ TO DO 1: do a bit exploration of other features on our own (5 pts)

TO DO: perform similar analysis to at least two other features of your choice. Use a text box to report your observations and understanding of these features.

Plotting graph between the area of the interior housing space that is above ground level and the number of floors built. This graph helps us understand if people prefer more number of floors if they can afford a large living area above ground. It can be inferred that most people prefer 2 floors even if the living space goes above 8000 sqft. It seems to be working as an inverse proportion between sqft_above and the number of floors with 2 floors being the mean (or the popular choice).

```
plt.scatter(data_without_id['floors'].values, data_without_id['sqft_above'].values)
plt.xlabel("floors")
plt.ylabel("sqft_above")
plt.show()
```



The heatmap of price as a function of view and zipcode describes whether the view of the building affect the price in a certain zipcode. From the plot, it is inferred that some zipcodes with lesser rate of the view also have houses with greater prices. The area with zipcode 98039 has a 0 view rate but yet has higher prices. It is also observed that the houses with view rate 4 in the same zipcode has the highest intensity of the prices. This implies that this particular zipcode is regionally expensive to live in regardless of the view.

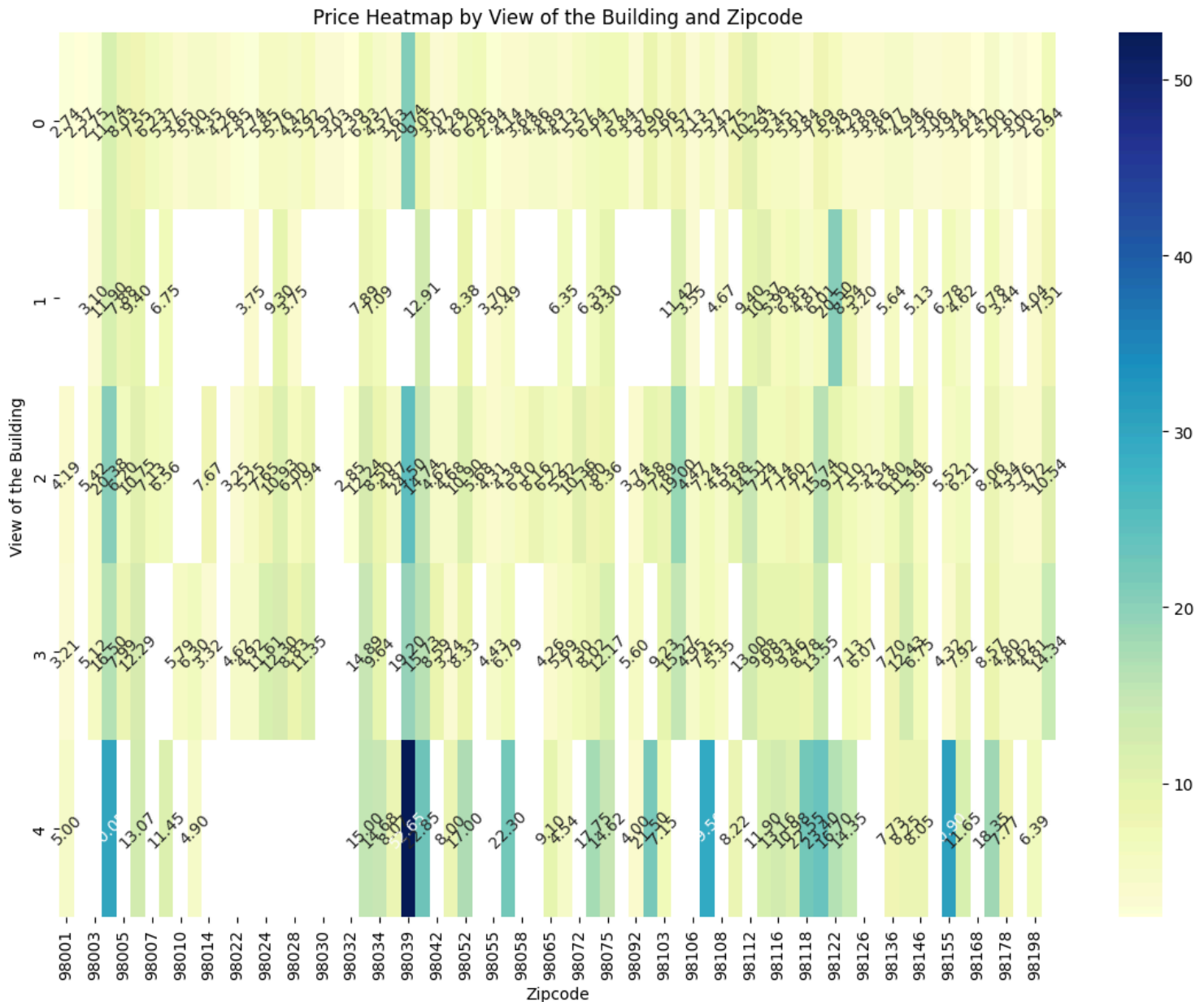
```
# put your code here for exploring other feautres. Feel free to use more blocks of text and code
# Create a pivot table to prepare data for the heatmap
pivot_table = data_without_id.pivot_table(index='view', columns='zipcode', values='price', aggfunc='mean')

# Create a heatmap using seaborn
plt.figure(figsize=(14, 10)) # Adjust the figure size if needed
heatmap = sns.heatmap(pivot_table, cmap='YlGnBu', annot=True, fmt='.2f', cbar=True)

for text in heatmap.texts:
    text.set(rotation=45)

# Add labels and a title to the plot
plt.xlabel('Zipcode')
plt.ylabel('View of the Building')
plt.title('Price Heatmap by View of the Building and Zipcode')

# Show the plot
plt.show()
```



✓ TO DO 2: handling categorical features (5 pts)

Many of the features appear to be numeric but in reality are of discrete nature --- in other words, they are more appropriately viewed as categorical variables. For example:

```
unique_zips = sorted(data_without_id['zipcode'].unique())
print(unique_zips)
```

